



UNIVERSITÀ DEGLI STUDI DI BERGAMO

Corso di studi in Tecnologie per l'Elettronica e l'Automazione (TEA)

CALCOLATORI ELETTRONICI · Filippo Ferrabue

University Student Information System

Progetto 1 · Linguaggio assemblativo MIPS

1. Dati studente

Anno Accademico: 2025/2026

Informatica: Modulo di Calcolatori Elettronici

Progetto Scelto: Progetto 1 – University Student Information System

Studente: Filippo Ferrabue

Matricola: 1106769

Appello: Sessione Giugno 2026

Indice:

1. Dati dello Studente e del Progetto	pag. 1
2. Introduzione e Architettura del Sistema	pag. 1
3. Analisi della funzione principale	pag. 4
4. Analisi Inizializzazione della Base Dati	pag. 6
5. Analisi delle funzioni	pag. 7
6. Analisi delle funzioni utility	pag. 23
7. Analisi delle macro	pag. 25
8. Robustezza del Sistema e Prevenzione dei Fatal Crash	pag. 26
9. Conclusioni e Limiti Architettureali	pag. 27

2. Introduzione e Architettura del Sistema

2.1 Descrizione Generale del Progetto

Il software sviluppato realizza un sistema informativo per la gestione delle carriere degli studenti universitari (*University Student Information System*) in linguaggio assemblativo **MIPS-32**, progettato per essere eseguito sull'emulatore **MARS 4.5**.

Il sistema modella un database in grado di ospitare fino a un massimo di **100 studenti**, associando a ciascuno di essi un libretto universitario elettronico che può contenere fino a un massimo di **20 esami superati**. L'interfaccia utente è interamente di tipo testuale ed è

governata da un menu interattivo centralizzato (locato nel ciclo principale `main_loop`) che permette di richiamare in modo modulare le diverse funzionalità del software tramite l'inserimento di un codice numerico (da 0 a 17).

2.2 Organizzazione della Memoria e Strutture Dati

A causa delle limitazioni native dell'architettura hardware MIPS, che non dispone del supporto a strutture dati complesse come oggetti o strutture (`struct`), l'intera base dati è stata ingegnerizzata nel segmento `.data` adottando due tecniche principali: gli **array paralleli** per le proprietà anagrafiche dello studente e le **matrici linearizzate** per la gestione bidimensionale degli esami.

Di seguito viene riportato il frammento di codice assembly esatto estratto dal file principale, che mostra l'allocazione statica della memoria tramite le direttive `.space` per configurare lo spazio massimo dei 100 studenti e dei relativi 2000 record d'esame totali:

```
# Studenti (max 100)
student_id:    .space 400    # 100 × 4 byte
student_name:  .space 2000   # 100 × 20 byte
student_surname: .space 2000 # 100 × 20 byte
student_age:   .space 400    # 100 × 4 byte
student_year:  .space 400    # 100 × 4 byte (anno iscrizione)
student_nexams: .space 400   # 100 × 4 byte (numero esami superati)
student_active: .space 400   # 100 × 4 byte (1=attivo, 0=eliminato)
student_count: .word 0       # numero studenti inseriti
search_buffer: .space 20     # spazio per inserire la parola da ricercare

# Esami (max 20 per studente, 100 studenti = 2000 totali)
exam_name:     .space 92000  # 2000 esami totali (100 stud. x 20) × 46 byte
exam_credits:  .space 8000   # 2000 × 4 byte
exam_grade:    .space 8000   # 2000 × 4 byte
```

2.2.1 Anagrafica degli Studenti (Array Paralleli)

Le informazioni di ogni singolo studente sono frazionate in vettori indipendenti e paralleli. La corrispondenza logica tra i dati dello stesso studente è garantita dall'uso del medesimo indice fisico `i` (compreso nel range da 0 a 99).

L'allocazione di memoria associata è così definita all'interno del segmento `.data`:

- `student_id` (100 studenti x 4 byte = 400 byte totali): Array di tipo Word contenente il codice identificativo numerico (ID) dello studente.
- `student_name` (100 studenti x 20 byte = 2000 byte totali): Spazio di memoria contiguo per stringhe ASCIIZ (massimo 19 caratteri memorizzabili più il terminatore nullo `\0`).
- `student_surname` (100 studenti x 20 byte = 2000 byte totali): Spazio per il cognome dello studente, configurato con una struttura analoga al nome.

- **student_age** (100 studenti x 4 byte = 400 byte totali): Array di Word per l'età anagrafica, validata a livello software per accettare solo valori compresi tra 18 e 100 anni.
- **student_year** (100 studenti x 4 byte = 400 byte totali): Array di Word indicante l'anno accademico di immatricolazione dello studente.
- **student_nexams** (100 studenti x 4 byte = 400 byte totali): Contatore di tipo Word che traccia quanti esami validi e superati ha registrato lo studente nel suo libretto.
- **student_active** (100 studenti x 4 byte = 400 byte totali): Flag di tipo Word (valore 1 per record attivo, valore 0 per record eliminato) utilizzato per implementare la rimozione logica veloce senza dover alterare o scalare l'allineamento degli array paralleli.

2.2.2 Libretto degli Esami (Matrici Linearizzate)

Per associare una lista di esami a uno studente senza sprecare puntatori dinamici o logiche di allocazione complesse, la memoria è stata organizzata calcolando l'offset in modo bidimensionale su una struttura sequenziale di 2000 elementi totali (dati dal prodotto di 100 studenti per 20 esami massimi).

La memoria complessiva degli esami si compone di tre matrici:

- **exam_name** (2000 elementi x 46 byte = 92000 byte totali): Matrice lineare di stringhe ASCII. Ogni singolo nome di esame occupa uno slot fisso di 46 byte (45 caratteri utili più il carattere di terminazione \0).
- **exam_credits** (2000 elementi x 4 byte = 8000 byte totali): Matrice lineare di tipo Word per memorizzare i crediti formativi (CFU) dei singoli esami.
- **exam_grade** (2000 elementi x 4 byte = 8000 byte totali): Matrice lineare di tipo Word contenente i voti in trentesimi conseguiti negli esami.

2.2.3 Formula di Linearizzazione e Calcolo degli Offset

Dato un indice studente **i** e un indice esame **k** (con k compreso tra 0 e il valore del contatore **student_nexams** relativo allo studente meno uno), la formula matematica per calcolare la posizione logica dell'esame all'interno del vettore monodimensionale è la seguente:

$$\text{Posizione Logica} = (i * 20) + k$$

Per tradurre questa posizione teorica in un indirizzo di memoria fisico espresso in byte (necessario per poter caricare o salvare i dati tramite le istruzioni hardware **lw**, **lb**, **sw**, **sb**), il codice calcola a runtime i seguenti offset di memoria:

- Per i dati strutturati a Word (come **exam_credits** ed **exam_grade**): **Offset in Byte = ((i * 20) + k) * 4** Questo calcolo è implementato nel codice assembly combinando l'istruzione di moltiplicazione **mul** e lo shift logico a sinistra **sll** di 2 posizioni per massimizzare la velocità di accesso.
- Per i dati strutturati a Stringa (come **exam_name**): **Offset in Byte = ((i * 20) + k) * 46**

2.3 Modularità del Codice e Struttura dei File

Il progetto rifiuta la logica dello sviluppo monolitico, distribuendo le routine e le macro su più file specializzati inclusi a tempo di compilazione tramite la direttiva **.include**. Questa scelta architetturale garantisce un'elevata leggibilità e isola in modo netto le componenti logiche del sistema.

Il pre-popolamento della base dati con un set iniziale di 50 record avviene attingendo alle dichiarazioni statiche allocate separatamente. Di seguito viene riportata a titolo di esempio l'organizzazione del segmento dati fisso per i primi record e per i relativi esami associati:

```
.data
# — STUDENTE o —
_db_nameo: .asciiz "Marco"
_db_surnameo: .asciiz "Rossi"
_db_ido: .word 1051
_db_ageo: .word 20
_db_yearo: .word 2022
_db_nexamso: .word 15
_db_activeo: .word 1
_db_eo_o_name: .asciiz "Analisi Matematica I"
_db_eo_o_cred: .word 9
_db_eo_o_grad: .word 28
_db_eo_1_name: .asciiz "Geometria e Algebra"
```

La scomposizione del software prevede sei file distinti:

1. **Ferrabue_progetto1.asm**: Costituisce il file principale e il punto di ingresso dell'applicazione (etichetta **main:**). Contiene l'allocazione primaria dello spazio in memoria nel segmento **.data**, la dichiarazione testuale del menu principale e la tabella di smistamento (**dispatch table**) che reindirizza l'esecuzione verso i vari casi.
2. **macros.asm**: Raccolta di macro globali riutilizzabili per snellire la scrittura del codice assembly. Include le routine standard di input e output (stampa di stringhe, lettura di interi e stringhe da tastiera) e la macro critica **print_student** dedicata all'astrazione visiva dei dati.
3. **database_studenti.asm**: File di dati statici puri che memorizza i record di 50 studenti pre-popolati con i relativi esami superati. Questo modulo è fondamentale per effettuare i test di carico e verificare la conformità degli algoritmi.
4. **inizializzazione.asm**: Contiene la logica di caricamento (eseguita al richiamo del Case o o come macro di avvio) che legge i dati dal file statico e popola fisicamente all'avvio i vettori allocati dinamicamente tramite le direttive **.space**.
5. **functions.asm**: Contiene il cuore applicativo del programma, ovvero i 17 macro-blocchi procedurali corrispondenti alle voci del menu (operazioni di inserimento, stampa, rimozione logica, calcoli statistici e ordinamento).
6. **utility_functions.asm**: Isola le funzioni di basso livello e di utilità generale per evitare la duplicazione del codice. Tra queste si distinguono la funzione **copy_string** (dotata di un blocco di sicurezza hardware anti-overflow a 19 caratteri), la routine **strip_newline** per la rimozione del carattere di invio nelle stringhe e la funzione matematica **get_student_gpa** per il calcolo della media ponderata.

2.4 Popolamento della Base Dati e Scenari di Test

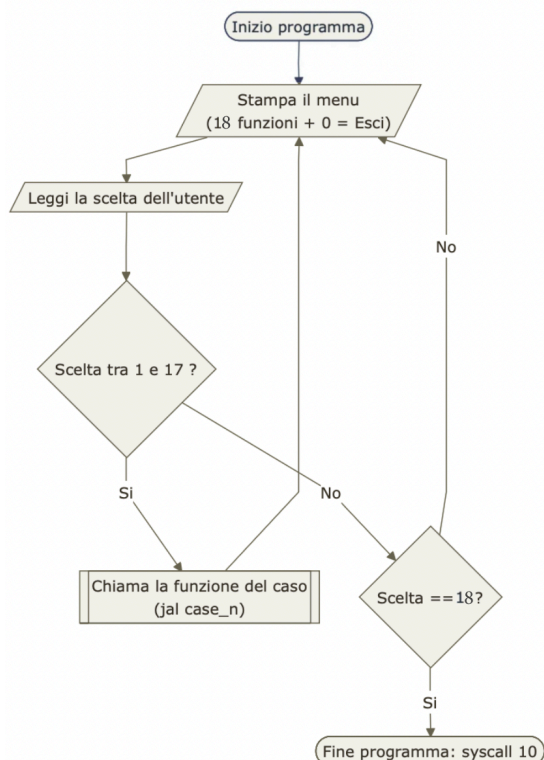
Come richiesto esplicitamente dalle specifiche del progetto, il popolamento iniziale del database non avviene in automatico, ma è associato all'**Opzione 0** del menù principale. Selezionando questa opzione, il sistema invoca la routine di inizializzazione che carica in memoria il set di 50 studenti pre-popolati nel file `database_studenti.asm`. Per agevolare il test delle funzioni, il set di dati include volutamente casi specifici pronti all'uso. Ad esempio, sono stati inseriti **studenti con cognomi uguali** (es. "Rossi") per testare la corretta restituzione di risultati multipli. Inoltre, sono già presenti **studenti non attivi** (`student_active = 0`) per dimostrare istantaneamente come il sistema ignori i record eliminati logicamente durante le stampe totali o i calcoli statistici. Infine, la presenza di libretti completamente vuoti (0 esami) o già saturi (20 esami) permette di verificare rapidamente i blocchi di sicurezza hardware e la prevenzione dei crash (es. la divisione per zero).

Questo set di dati simula una casistica reale e include intenzionalmente dei "casi limite" (*edge cases*) per dimostrare la stabilità degli algoritmi durante la verifica delle 17 funzionalità:

- **Quantità e Range ID:** Sono presenti **50 studenti** pronti per essere caricati. I codici identificativi (ID) vanno **da 1001 a 1051** (lo Studente 0, Marco Rossi, ha ID 1051).
- **Distribuzione degli Esami e Materie:** I libretti simulano corsi reali come *Analisi Matematica I*, *Fisica I*, *Elettronica* e *Calcolatori Elettronici*, con crediti (6, 9, 12 CFU) e voti (da 18 a 30) già assegnati per verificare i calcoli matematici.

3. Logica Algoritmica, Analisi dei Flussi e Implementazione Analitica

In questo capitolo il software viene analizzato in singoli blocchi funzionali. Ciascuna procedura viene esaminata nel dettaglio attraverso diagrammi di flusso (**flow-chart**) e spiegazioni tecniche del rispettivo codice Assembly.



3.1 Funzione Principale e Ciclo di Controllo (Main & Main Loop)

Il punto di ingresso del programma gestisce l'inizializzazione del sistema e l'esecuzione del ciclo infinito che mostra il menu interattivo e smista le richieste dell'utente.

```

main_loop: li $v0, 4
la $a0, menu_msg
syscall
li $v0, 5
syscall
move $t0, $v0 # $t0 = scelta utente

```

Mostra a video l'elenco delle opzioni tramite la `syscall 4`, acquisisce l'input numerico dell'operatore con la `syscall 5` e lo copia nel registro temporaneo `$t0`.

```

li $t1, 1
beq $t0, $t1, case1
# ... [casi intermedi da 2 a 17] ...
beq $t0, $zero, case0

```

```
j main_loop      # scelta non valida → rimostra il menù
```

Dispatch Table (*switch-case*): confronta sequenzialmente l'input in `$t0` con i codici dei casi e devia il flusso. L'istruzione `j main_loop` fa da fallback per gli input errati.

```
case1:
    jal insert_student
    j main_loop
```

Infrastruttura standard dei casi: la chiamata `jal` esegue la sotto-routine esterna salvando l'indirizzo di ritorno, mentre `j main_loop` garantisce il rientro continuo al ciclo.

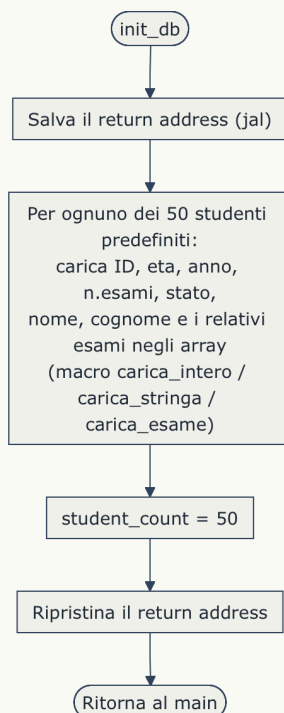
```
case0:
    li $v0, 10
    syscall
```

Gestisce l'uscita ordinata dall'applicazione invocando la routine di terminazione hardware tramite la `syscall 10`

```
.include "functions.asm"
.include "inizializzazione.asm"
.include "utility_functions.asm"
```

Inclusioni modulari poste in coda per permettere al compilatore di linkare correttamente tutte le funzioni esterne richiamate dai vari casi del ciclo principale.

3.2 Inizializzazione della Base Dati (init_db)



Selezionando l'opzione 0 dal menù principale, il processore esegue una chiamata a procedura (`jal init_db`). Questa routine si occupa di leggere i record allocati staticamente nel file esterno `database_studenti.asm` e di trascriverli all'interno degli spazi di memoria allocati tramite le direttive `.space` nel segmento `.data` principale del programma. Il ciclo viene eseguito per 50 volte, incrementando il contatore globale `student_count` e impostando inizialmente a 1 il flag `student_active` per ciascun record caricato.

Analisi Strutturale del Codice Sorgente

```
init_db:
    li $t0, 0          # i = 0
    li $t1, 50         # Limite record del database
```

Azzera l'indice di scansione `$t0` e imposta a 50 il registro di controllo `$t1`, definendo il numero esatto di record precaricati da importare.

Plaintext

```
init_loop:
    beq $t0, $t1, init_done  # Esci se i == 50
    sll $t2, $t0, 2          # Offset per elementi Word (i * 4)
```

Verifica la condizione di terminazione del ciclo. Se il test fallisce, calcola l'offset di memoria moltiplicando l'indice per 4 byte tramite uno shift logico a sinistra (**sll**).

```
lw $t3, db_id($t2)      # Legge ID da sorgente statica
sw $t3, student_id($t2)  # Scrive ID in array di lavoro
li $t4, 1
sw $t4, student_active($t2) # Forza flag attivo = 1
```

Estrae il codice identificativo dal database pregenerato e lo copia nel rispettivo array dinamico. Contestualmente, inizializza a 1 il flag nel vettore **student_active** per validare il record. Questa logica di lettura/scrittura viene replicata per tutti i restanti campi anagrafici.

```
addi $t0, $t0, 1      # i++
j init_loop
init_done:
jr $ra                # Ritorno al chiamante
```

Incrementa l'indice iterativo per passare allo studente successivo e reitera il ciclo. Al termine dei 50 caricamenti, l'istruzione **jr \$ra** restituisce il controllo alla funzione principale **main**.

Tabella 3.3 – Struttura logica delle funzioni

Nota descrittiva sulla corrispondenza tra opzioni del menù, routine in Assembly MIPS e documentazione grafica dei flussi.

#	Funzione MIPS	Descrizione
0	init_db	
1	insert_student	Inserisci un nuovo studente (con i suoi esami)
2	print_all_students	Stampa tutti gli studenti attivi
3	search_by_id	Cerca uno studente per ID
4	search_by_surname	Cerca studenti per cognome
5	add_exam	Aggiungi un esame a uno studente
6	edit_student	Modifica i dati di uno studente
7	delete_student	Elimina (logicamente) uno studente
8	calculate_gpa	Media ponderata di uno studente
9	compute_best_student	Miglior studente per media ponderata
10	compute_total_credits	Crediti totali acquisiti
11	print_students_above_threshold	Studenti sopra una soglia di media
12	print_students_min_exams	Studenti con almeno N esami

13	sort_students_by_id	Ordina il database per ID
14	sort_students_by_gpa	Ordina il database per media
15	print_statistics	Statistiche del database
16	export_compact_report	Report compatto (una riga per studente)
17	recursive_print_exams	Lista esami con stampa ricorsiva
18	syscall 10	Esci dal programma

3.3.1 Caso 1: Inserimento Nuovo Studente (insert_student)

Questa funzione gestisce l'aggiunta di un nuovo record validando la capienza della memoria e l'integrità dei dati anagrafici.

Analisi Strutturale del Codice Sorgente

```
insert_student:  
  addi $sp, $sp, -4  
  sw $ra, o($sp)  
  la $to, student_count  
  lw $t1, o($to)      # $t1 = numero studenti  
  li $t2, 100
```

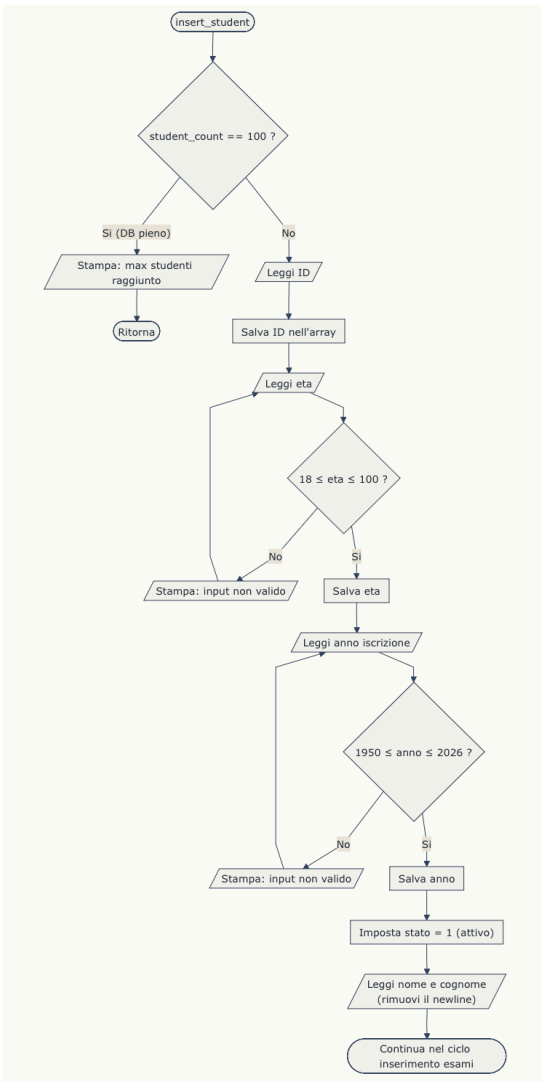
```
  beq $t1, $t2, db_full_error
```

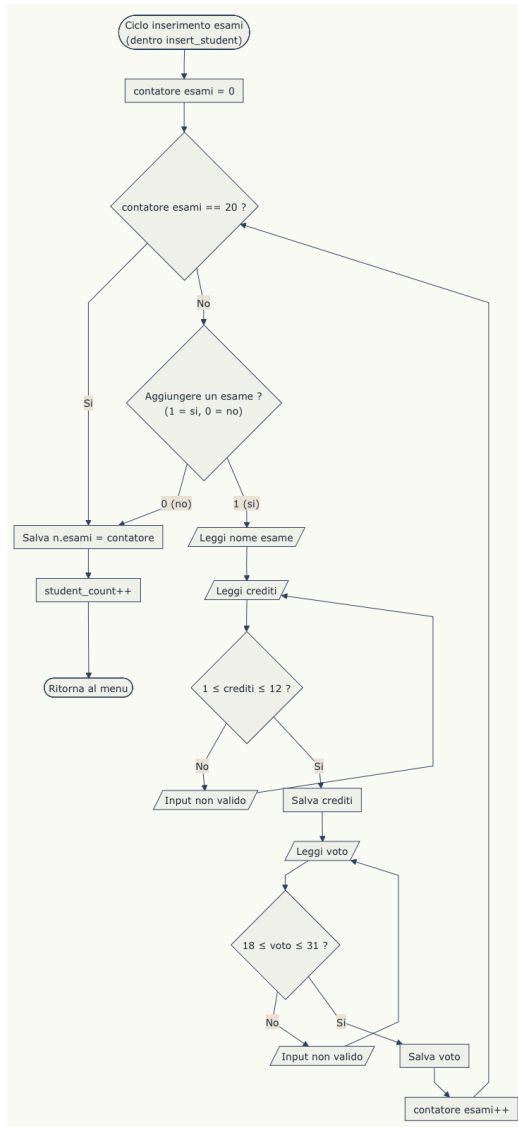
Salva l'indirizzo di ritorno allocando spazio nello Stack (\$sp). Estrae il numero di studenti dal contatore e verifica la capienza massima (100). In caso di saturazione, blocca l'inserimento e devia all'errore.

```
sll $t3, $t1, 2      # Calcolo offset Word (i * 4)  
li $t4, 20  
mul $t5, $t1, $t4    # Calcolo offset Stringhe (i * 20)
```

Calcola dinamicamente gli indirizzi di memoria. Tramite shift logico (sll) moltiplica l'indice per 4 byte (per vettori di interi come ID ed età), mentre con mul lo moltiplica per 20 byte (per i testi di nome e cognome).

```
chiedi_intero(insert_id_msg, $t6)  
la $t7, student_id  
add $t7, $t7, $t3  
sw $t6, o($t7)
```





Sfrutta la macro `chiedi_intero` per acquisire l'ID. Somma l'offset calcolato (`$t3`) all'indirizzo base dell'array `student_id` e salva fisicamente la Word in memoria tramite `sw`. Questo processo si ripete per l'acquisizione di nome, cognome e anno.

`ask_age:`

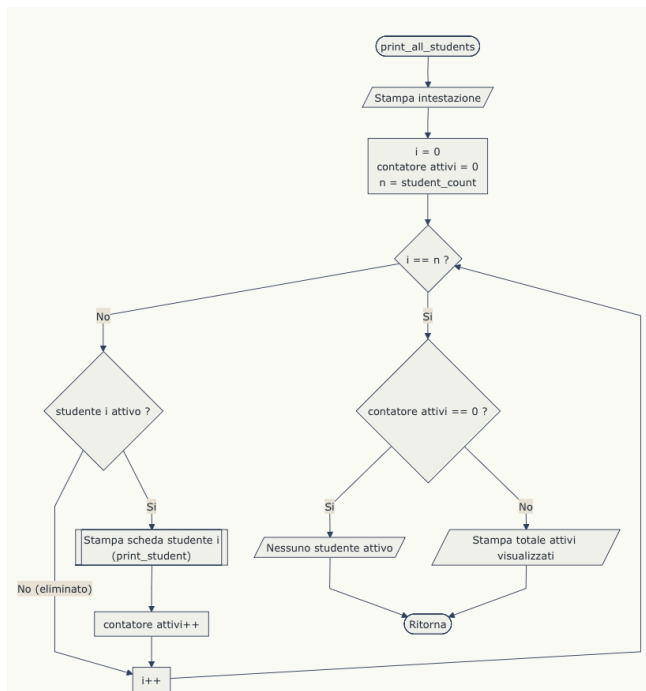
```
chiedi_intero(insert_age_msg, $t6)
bgt $t6, 100, invalid_age
blt $t6, 18, invalid_age
sw $t6, student_age($t3)
```

Valida in hardware l'età inserita: la logica condizionale scarta i valori maggiori di 100 (`bgt`) o minori di 18 (`blt`). Solo se il test è superato, sovrascrive il dato nell'array `student_age`.

```
addi $t1, $t1, 1
sw $t1, o($t0)      # student_count++
lw $ra, o($sp)
addi $sp, $sp, 4
jr $ra
```

A inserimento completato, incrementa il contatore globale degli studenti, ripristina l'indirizzo di ritorno estraendolo dallo Stack e dealloca i byte usati (`addi $sp, $sp, 4`), restituendo infine il controllo al ciclo principale.

3.3.2 Caso 2: Stampa Elenco Studenti Attivi (`print_all_students`)



La routine esegue una scansione lineare della memoria per mostrare a video i dati anagrafici dei soli studenti che risultano validi all'interno del sistema.

Analisi Strutturale del Codice Sorgente

`print_all_students:`

```
addi $sp, $sp, -4
sw $ra, o($sp)
li $t0, 0          # i = 0 (Indice del ciclo)
la $t1, student_count
lw $t1, o($t1)     # $t1 = Numero totale studenti inseriti
```

Preserva l'indirizzo di ritorno nello Stack. Inizializza il registro `$t0` a zero come contatore del ciclo e carica

in `$t1` il numero totale di record presenti per definire il limite della scansione lineare.

```
print_all_loop:
    beq $t0, $t1, print_all_end # Condizione di terminazione: i == student_count
    sll $t2, $t0, 2             # Calcolo offset per vettori di Word (i * 4)
    lw $t3, student_active($t2) # Caricamento del flag di stato
    beq $t3, $zero, next_print # Se il flag è 0 (Record eliminato), salta alla cella successiva
```

Controlla il raggiungimento della fine della base dati. Se il ciclo continua, ricava l'offset a 32 bit per estrarre il flag dal vettore `student_active`. In caso di record disattivato logicamente (*soft-delete*), devia il flusso saltando la stampa.

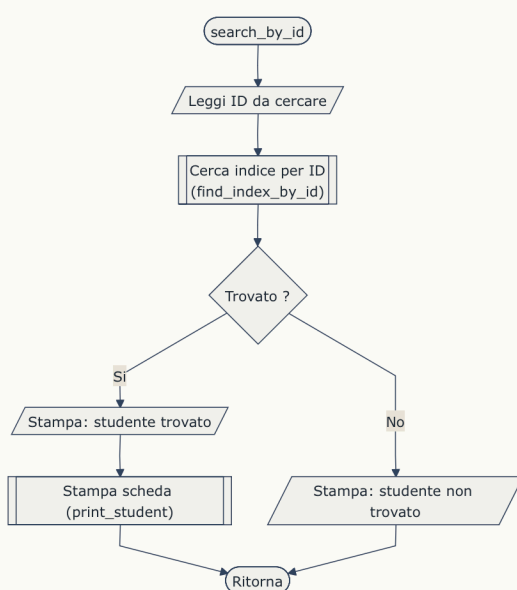
```
print_student($t0)    # Invocazione della macro globale di stampa
next_print:
    addi $t0, $t0, 1    # Incremento dell'indice del ciclo (i++)
    j print_all_loop    # Salto incondizionato alla testa del ciclo
```

Se il record è attivo (flag pari a 1), invoca la macro `print_student` passando l'indice corrente per formattare e inviare i dati ai servizi di sistema. Successivamente incrementa l'indice e reitera il ciclo.

```
print_all_end:
    lw $ra, 0($sp)
    addi $sp, $sp, 4    # Ripristino dello Stack Pointer
    jr $ra              # Ritorno alla routine chiamante (main_loop)
```

Raggiunto il limite di scansione, estrae il vecchio indirizzo di collegamento memorizzato in precedenza nello Stack, ripristina lo spazio di memoria della CPU e restituisce il controllo al gestore del menù principale.

3.3.3 Caso 3: Ricerca per Codice ID (search_by_id)



La routine acquisisce l'ID e sfrutta una macro centralizzata per invocare la ricerca di backend, riducendo le duplicazioni di codice nel file sorgente.

```
search_by_id:
    addi $sp, $sp, -4
    sw $ra, 0($sp)
    chiedi_intero(search_id_msg, $t0)
```

Salva il registro di ritorno nello stack e sfrutta la macro di I/O `chiedi_intero` per stampare la richiesta a video e acquisire l'ID numerico inserito dall'utente, depositandolo nel registro `$t0`.

```
cerca_studente_per_id($t0,
print_id_not_found)
```

```
move $s0, $v0
```

Invoca la macro architetturale `cerca_studente_per_id`. Questa macro trasferisce l'ID al motore di ricerca lineare e gestisce autonomamente il salto di errore se l'ID non esiste. Se il record viene trovato, l'indice di memoria valido restituito in `$v0` viene messo al sicuro in `$s0`.

```
li $v0, 4
la $a0, found_msg
syscall
move $a0, $s0
print_student($a0)
j search_id_end
```

Conferma il successo della ricerca a video tramite la `syscall 4`. Subito dopo, sposta l'indice in `$a0` e chiama la macro globale `print_student` per estrarre e formattare i dati anagrafici a schermo. L'istruzione `j` salta la gestione degli errori per puntare all'epilogo.

```
print_id_not_found:
li $v0, 4
la $a0, not_found_msg
syscall
```

Blocco di fallback *catch*. Raggiunto solo se la macro di ricerca rileva l'assenza del record o incontra un flag di stato inattivo. Mostra il messaggio di fallimento senza interrompere il ciclo di vita del software.

```
search_id_end:
lw $ra, o($sp)
addi $sp, $sp, 4
jr $ra
```

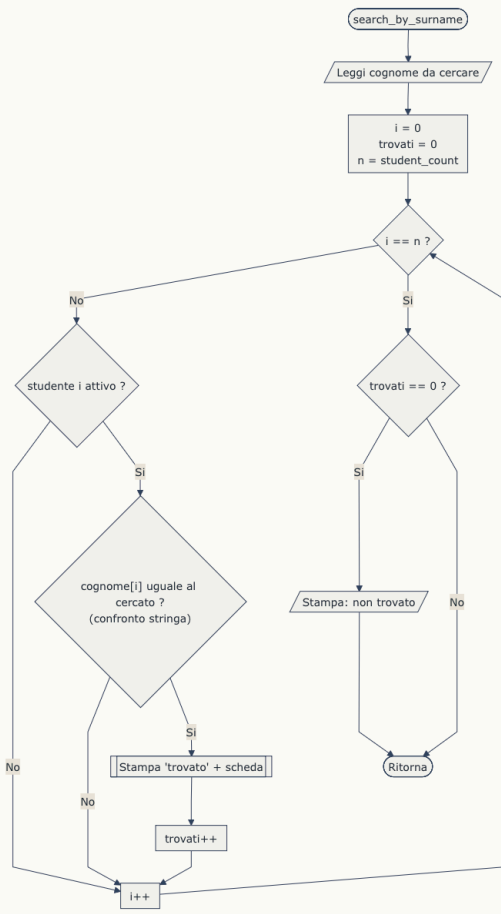
Epilogo unificato. Ripristina l'indirizzo salvato nello Stack Pointer ed esegue il rientro incondizionato al menù principale del ciclo.

3.3.4 Caso 4: Ricerca per Cognome (search_by_surname)

Esegue un filtraggio testuale ricercando le occorrenze esatte di una stringa alfabetica, implementando un confronto byte a byte sui vettori di caratteri.

Analisi Strutturale del Codice Sorgente

```
search_by_surname:
addi $sp, $sp, -4
sw $ra, o($sp)
chiedi_stringa(search_surname_msg, search_buffer, 20)
```



la \$a0, search_buffer
jal strip_newline

Salva l'indirizzo di ritorno e utilizza la macro **chiedi_stringa** per acquisire l'input testuale dell'utente, depositandolo nel **search_buffer**. Immediatamente dopo, invoca l'utility **strip_newline** per rimuovere il carattere di invio (Enter/\n) dalla stringa, garantendo un confronto esatto in memoria.

search_surname_loop:

```

beq $t0, $t1, search_surname_end
sll $t2, $t0, 2      # Offset Word (i * 4)
lw $t3, student_active($t2)
beq $t3, $zero, next_surname

```

Avvia la scansione lineare sul contatore di studenti. Calcola l'offset a 32-bit e verifica preliminarmente il flag **student_active**. Se il record è disattivato

logicamente, salta direttamente alla cella successiva omettendo i calcoli sulle stringhe.

```

la $a0, search_buffer    # Indirizzo della parola cercata
li $t5, 20
mul $t4, $t0, $t5        # Offset Stringhe (i * 20)
la $a1, student_surname
add $a1, $a1, $t4        # Indirizzo del cognome corrente
jal compare_strings

```

Allinea i registri argomento per la comparazione. Carica in **\$a0** il buffer di ricerca e in **\$a1** l'indirizzo del cognome in memoria, ricavato moltiplicando l'indice per 20 byte (**mul**). Infine, devia il flusso verso la sotto-routine **compare_strings** per l'analisi dei singoli caratteri.

```

beq $v0, $zero, surname_found
next_surname:
addi $t0, $t0, 1
j search_surname_loop

```

Al rientro dalla funzione, analizza l'output depositato in **\$v0**. Se è uguale a 0 (corrispondenza trovata), il branch **beq** devia l'esecuzione al blocco di stampa (**surname_found**) che richiama la macro **print_student**. Altrimenti, incrementa l'indice iterativo e riavvia il loop.

```
search_surname_end:
```

```
lw $ra, o($sp)
addi $sp, $sp, 4
jr $ra
```

Epilogo della funzione. Terminata la scansione dell'array, ripristina lo Stack Pointer prelevando il valore originale di **\$ra** e restituisce il controllo al *Main Loop*.

3.3.5 Caso 5: Aggiunta Esame al Libretto (add_exam)

Registra un nuovo esame all'interno della matrice linearizzata, verificando i limiti di capienza e prevenendo la duplicazione dei corsi.

Analisi Strutturale del Codice Sorgente

```
cerca_studente_per_id($to, add_exam_not_found)
move $so, $vo          # Salva l'indice in $so
# [estrazione n_esami in $s1]
li $t3, 20
beq $s1, $t3, add_exam_full # Errore se max 20 esami
```

Sfrutta la macro centralizzata per localizzare l'ID. Una volta trovato l'indice, estrae il numero di esami già sostenuti e blocca la procedura se il libretto ha raggiunto la capienza massima consentita (20 slot).

```
mul $t4, $so, $t3      # Indice linearizzato = (studente * 20)
add $t4, $t4, $s1      # + indice esame corrente
# [calcolo byte e check_duplicate_loop]
beq $t7, $zero, duplicate_found
```

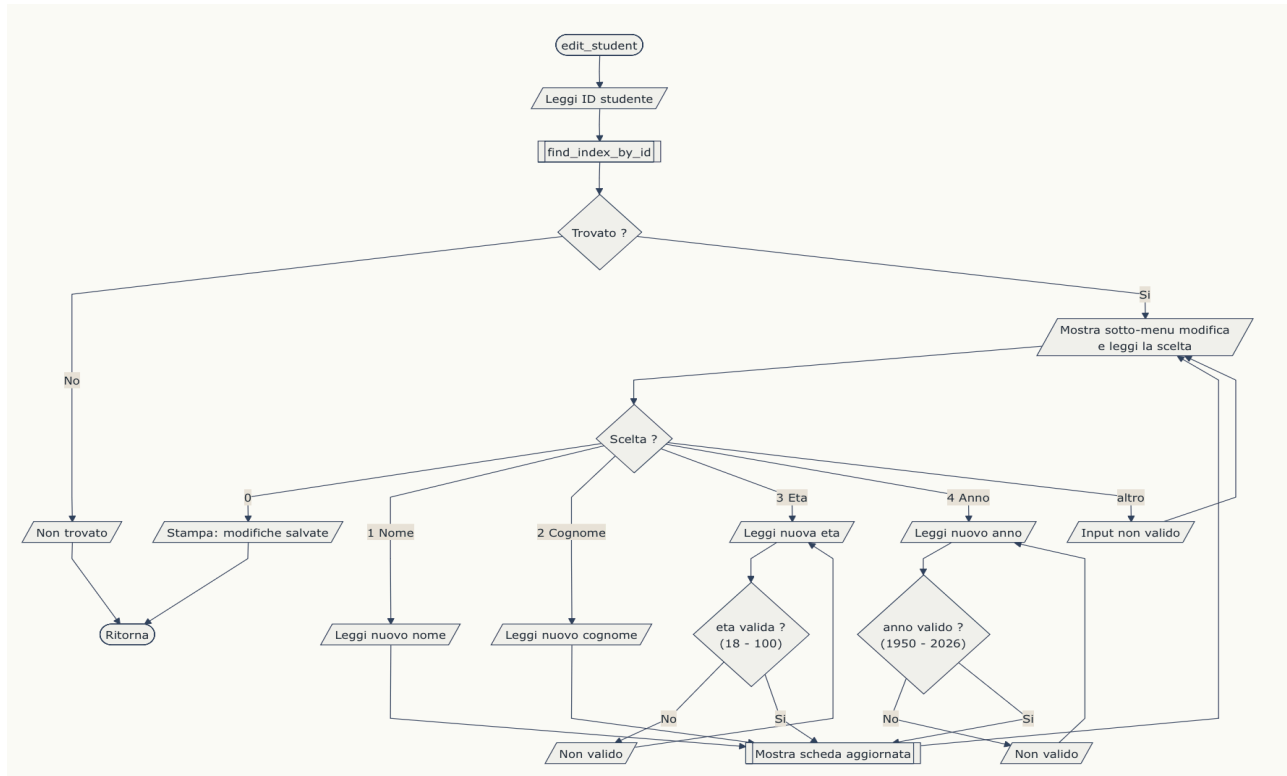
Linearizza gli indirizzi della matrice bidimensionale per trovare la cella esatta di inserimento. Prima della scrittura, un ciclo di parsing scansiona lo storico e interrompe l'operazione se il nuovo esame ha lo stesso nome di uno già registrato.

```
# [acquisizione e validazione CFU e Voto]
```

```
sw $t8, o($t7)         # Scrittura definitiva in memoria
addi $s1, $s1, 1       # n_esami++
sw $s1, o($t2)         # Aggiorna il contatore nel libretto
```

Dopo aver acquisito e validato i dati (es. CFU tra 1 e 12), sovrascrive le Word negli indirizzi calcolati e aggiorna in via definitiva il contatore globale degli esami dello studente.

3.3.6 Caso 6: Modifica Dati Anagrafici (edit_student)



Permette l'aggiornamento selettivo dei singoli attributi di uno studente tramite un sotto-menù di instradamento, riutilizzando le macro di ricerca per localizzare e validare il record.

Analisi Strutturale del Codice Sorgente

```

chiedi_intero(search_id_msg, $to)
cerca_studente_per_id($to, edit_not_found)
move $so, $vo      # $so = Indice studente
  
```

Acquisisce l'identificativo bersaglio e invoca la macro architetturale di ricerca. Se l'ID non è presente in memoria o appartiene a un record disattivato, devia alla routine di errore; altrimenti, salva l'indice di memoria valido nel registro **\$s0**.

```

edit_dispatch:
    beq $to, $zero, edit_exit
    li $t1, 1
    beq $to, $t1, edit_name
    # [casi per cognome e anno di iscrizione]
    li $t1, 3
    beq $to, $t1, edit_age
  
```

Acquisita la scelta di modifica, implementa una *Dispatch Table* locale. Tramite una cascata di branch condizionali (**beq**), instrada il flusso di esecuzione verso la specifica sotto-routine incaricata di manipolare la singola variabile anagrafica.

```
edit_name:
    # [acquisizione e strip_newline]
    li $t1, 20
    mul $t2, $s0, $t1      # Offset (i * 20)
    la $a1, student_name
    add $a1, $a1, $t2
    jal copy_string
```

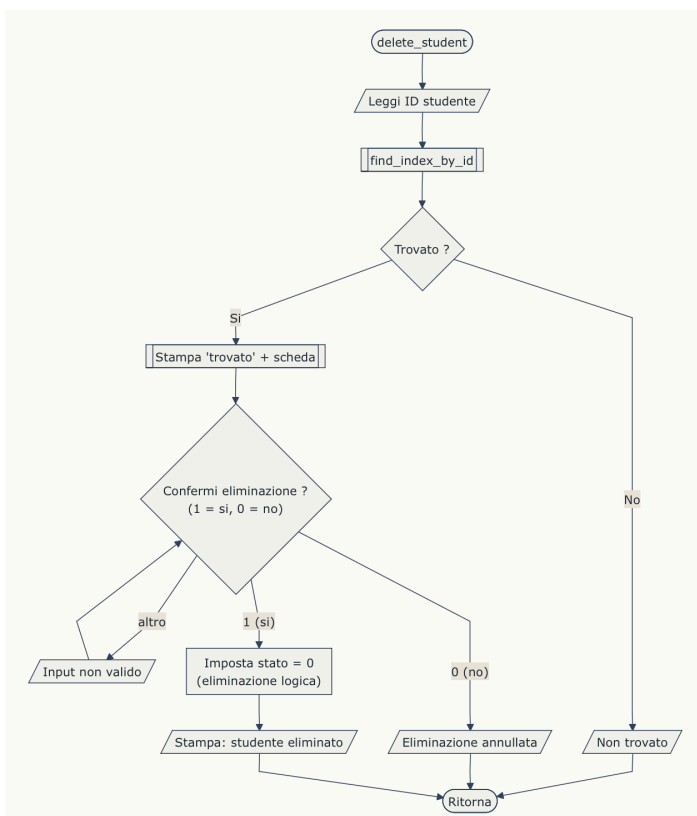
Nel caso di variabili testuali, calcola l'offset a 20 byte moltiplicando l'indice tramite **mul**. Dopo aver ripulito l'input dal terminatore di invio, invoca la routine di utility **copy_string** per sovrascrivere i caratteri nel buffer di destinazione.

```
edit_age:
    chiedi_intero(insert_age_msg, $t3)
    bgt $t3, 100, edit_age_err # Protezione limite superiore
    blt $t3, 18, edit_age_err  # Protezione limite inferiore
    sw $t3, student_age($t2)   # Sovrascrittura Word in memoria
```

Nelle modifiche numeriche, la routine reitera i medesimi controlli hardware di range [18-100] utilizzati nell'inserimento per prevenire corruzioni logiche. Solo se il dato è valido, l'istruzione **sw** sovrascrive fisicamente la Word obsoleta.

3.3.7 Caso 7: Cancellazione Logica (delete_student)

Implementa la rimozione di un record tramite strategia di soft-delete, disattivando lo studente senza alterare la struttura fisica delle matrici in memoria.



Analisi Strutturale del Codice Sorgente

```
chiedi_intero(search_id_msg, $t0)
cerca_studente_per_id($t0,
delete_not_found)
    move $s0, $v0      # $s0 = Indice
                        # studente trovato
```

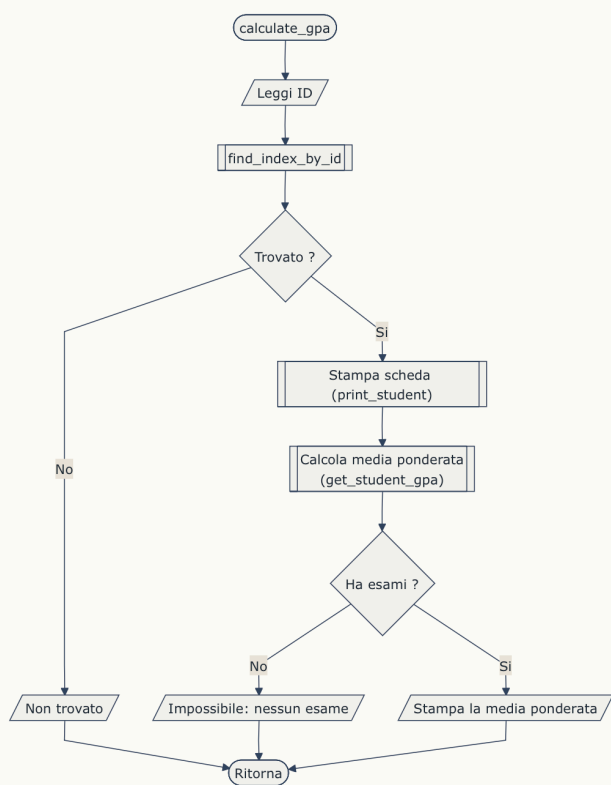
Come per le funzioni precedenti, delega alle macro centralizzate l'acquisizione dell'identificativo e la ricerca lineare nel database. Se la validazione va a buon fine, l'indice di memoria valido viene spostato nel registro **\$s0**.

```
sll $t1, $s0, 2      # Offset Word (i * 4)
    sw $zero, student_active($t1) #
Sovrascrittura del flag a 0
```


Anziché traslare pesantemente l'intera memoria riordinando gli array con un algoritmo $O(N)$ di shifting, la routine esegue un'operazione istantanea $O(1)$: calcola l'offset e forza a zero la Word associata nel vettore `student_active`. I dati storici rimangono intatti, ma lo studente verrà automaticamente ignorato da tutte le successive operazioni di ricerca, calcolo e stampa.

3.3.8 Caso 8: Calcolo Media Ponderata (calculate_gpa)

Richiede l'ID di uno studente, richiama il motore di calcolo matematico in *backend* e stampa il risultato sfruttando il co-processore per la virgola mobile (FPU).



Analisi Strutturale del Codice Sorgente

```

chiedi_intero(search_id_msg, $to)
        cerca_studente_per_id($to,
print_gpa_not_found)
        move $so, $vo          # $so = Indice
                                studente trovato
  
```

Acquisisce l'ID e sfrutta la macro centralizzata per validare l'esistenza del record. Se lo studente è attivo, memorizza l'indice valido in `$s0`.

```

move $ao, $so
jal get_student_gpa    # Ritorna Media
in $fo e Stato in $v1
beq $v1, $zero, print_gpa_no_exams
  
```

Passa l'indice come argomento e invoca la *utility* esterna `get_student_gpa` delegando il calcolo matematico. Al rientro, verifica il flag di stato `$v1`: se vale zero

(nessun esame a libretto), la condizione `beq` devia il flusso interrompendo la procedura e prevenendo un errore hardware di divisione per zero.

```

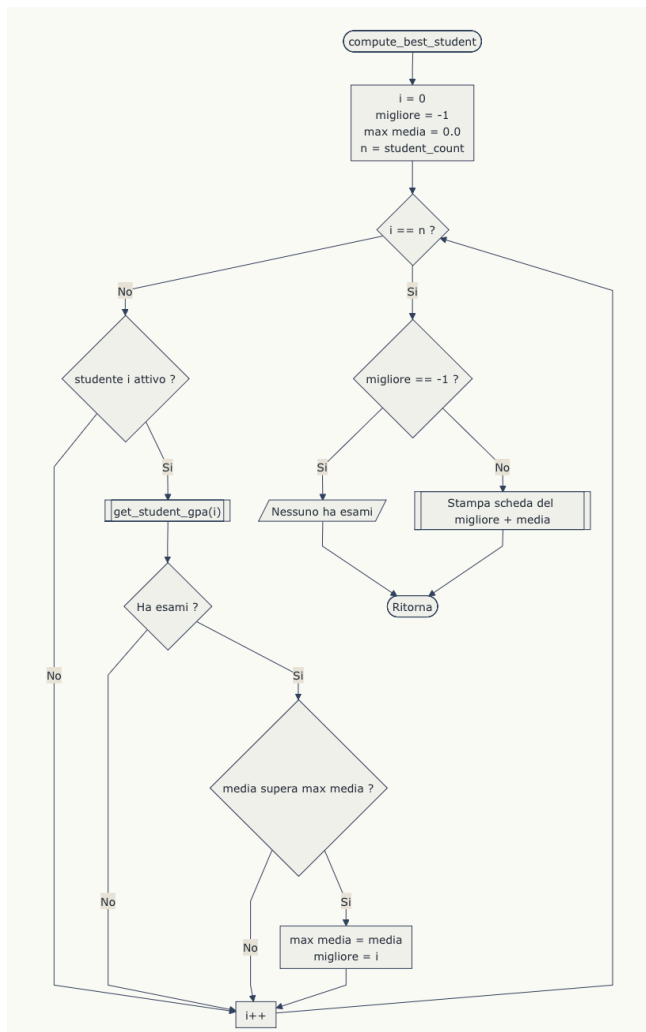
mov.s $f12, $fo      # Spostamento nel registro Float di I/O
li $vo, 2            # Syscall per la stampa di Float (Singola Precisione)
syscall
  
```

Gestisce l'output in virgola mobile comunicando direttamente con l'FPU (*Floating Point Unit*). Tramite l'istruzione dedicata `mov.s`, trasferisce il risultato calcolato dal registro

coprocessore `$f0` al registro argomenti `$f12`, invocando infine la `syscall 2` per la formattazione a schermo.

Ecco l'analisi per il **Caso 9 (Studente Migliore)**. Qui entra in gioco l'istruzione di branch condizionale legata al coprocessore matematico, un dettaglio tecnico che valorizza molto la documentazione.

3.3.9 Caso 9: Studente Migliore (compute_best_student)



La funzione scansiona l'intero database calcolando dinamicamente le medie e sfruttando i flag hardware dell'unità in virgola mobile per individuare l'eccellenza accademica.

Analisi Strutturale del Codice Sorgente

```

mtc1 $zero, $f12      # Inizializza il Max_GPA a
0.0 nel coprocessore FPU
li $s2, -1            # Inizializza l'indice del migliore a
-1
best_student_loop:
# [Omissis controlli fine loop e check flag active]
move $a0, $t0
jal get_student_gpa    # Ritorna Media corrente
in $f0 e Stato in $v1
  
```

Prepara l'ambiente azzerando il registro Float `$f12` (che conterrà la media massima) spostando i bit dal registro `$zero` della CPU principale al co-processore tramite l'istruzione dedicata `mtc1` (*Move To Coprocessor 1*). Nel ciclo, invoca la routine matematica per calcolare istantaneamente il GPA del record analizzato.

```

beq $v1, $zero, next_best_student # Salta gli
studenti senza esami a libretto
c.le.s $f0, $f12                  # (FPU) Conpara:
Media_Corrente <= Max_Provvisorio?
  
```

```
bc1t next_best_student    # Branch on Coprocessor 1 True
```

Scarta preventivamente gli studenti senza esami controllando il flag di stato `$v1`. Successivamente, delega il confronto decimale all'FPU tramite l'istruzione `c.le.s` (*Compare Less or Equal Single*). Il salto condizionato `bc1t` valuta il flag matematico appena impostato: se la media corrente è inferiore o uguale al massimo provvisorio, la CPU salta direttamente all'iterazione successiva.

```

mov.s $f12, $f0          # Aggiorna il nuovo GPA massimo
move $s2, $t0            # Salva l'indice anagrafico del nuovo migliore
  
```

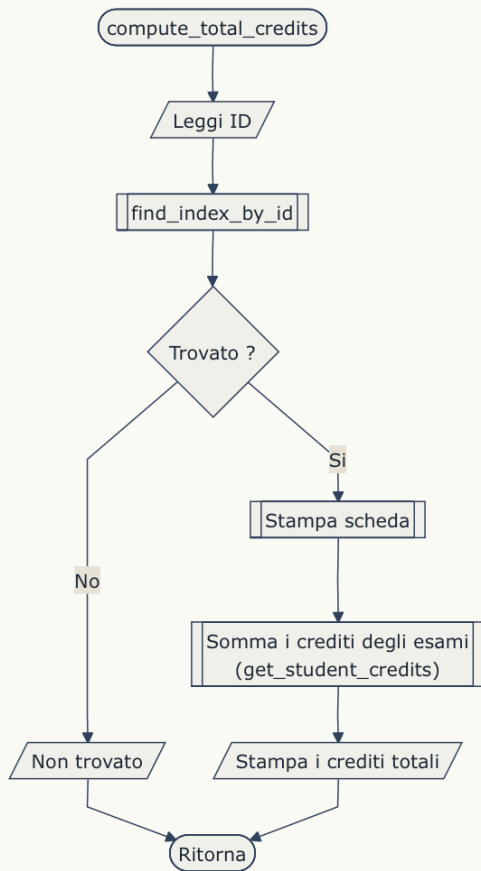
```

next_best_student:
    addi $t0, $t0, 1      # Incremento indice i++
    j best_student_loop

```

Se il *branch* FPU non viene eseguito, significa che il sistema ha rilevato un nuovo record battente. L'istruzione **mov.s** sovrascrive il picco massimo in virgola mobile e il registro **\$s2** memorizza l'indice del vincitore. A fine ciclo, il registro **\$s2** verrà fornito alla macro di stampa per mostrare l'anagrafica completa.

3.3.10 Caso 10: Calcolo Crediti Totali (compute_total_credits)



Determina la somma complessiva dei Crediti Formativi Universitari (CFU) acquisiti da un singolo studente, iterando sulla riga corrispondente all'interno della matrice degli esami.

Analisi Strutturale del Codice Sorgente

```

chiedi_intero(search_id_msg, $t0)
cerca_studente_per_id($t0, print_cred_not_found)
move $a0, $v0      # Passa l'indice come argomento
jal get_total_credits  # Invoca la routine di utility

```

La funzione principale gestisce l'interazione con l'utente: acquisisce l'ID, ne verifica l'esistenza in memoria tramite macro e, in caso di successo, passa l'indice di memoria valido al registro argomento **\$a0** delegando il calcolo alla sotto-routine di *backend*.

```

# [Dentro la utility]
lw $s2, 0($t1)      # $s2 = numero di esami da
                    # scansionare
li $s1, 0           # $s1 = Accumulatore Crediti
li $t9, 0           # $t9 = Indice ciclo esami (k)

```

La *utility* prepara l'ambiente di calcolo: estrae il tetto massimo del ciclo (il numero di esami effettivamente registrati a libretto), azzerava l'accumulatore delle somme e inizializza a zero il contatore dell'iterazione.

```

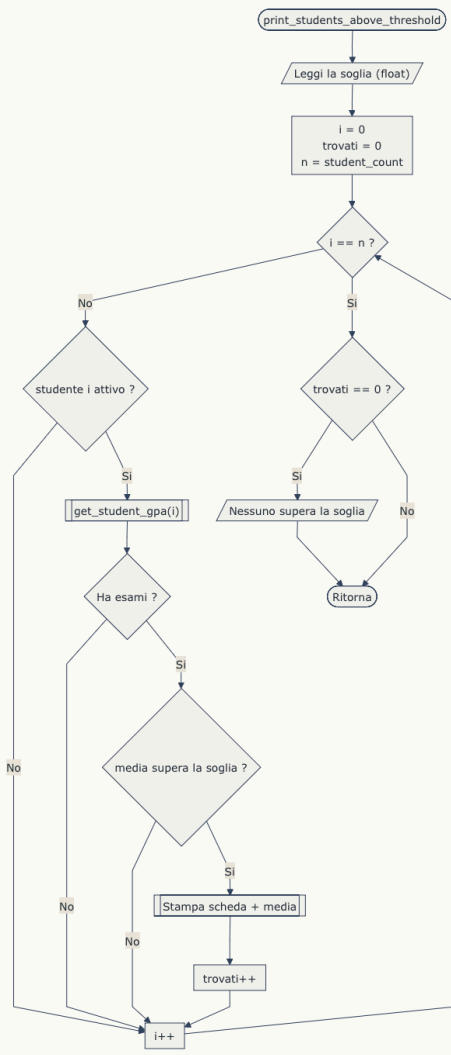
get_credits_loop:
    beq $t9, $s2, get_credits_exit # Condizione d'uscita
    # [Omissis calcolo offset: ((i * 20) + k) * 4]
    la $t0, exam_credits
    add $t0, $t0, $t2
    lw $t3, 0($t0)      # $t3 = CFU esame corrente
    add $s1, $s1, $t3   # Somma: Accumulatore += CFU

```

Esegue un ciclo definito. Per ogni esame, ricalcola dinamicamente l'offset linearizzato della matrice estraendo la Word corrispondente dal vettore **exam_credits**. Il valore estratto

viene sommato iterativamente all'accumulatore `$s1`. Al termine del ciclo, il totale viene restituito al chiamante per essere stampato a schermo tramite `syscall`.

3.3.11 Caso 11: Filtro per Soglia di Media (`print_students_above_threshold`)



La routine acquisisce un valore e si interfaccia con l'unità FPU per confrontare dinamicamente il GPA di ogni studente, stampando solo le eccellenze.

Analisi Strutturale del Codice Sorgente

```

# [Omissis acquisizione float della soglia utente in $f2]
threshold_loop:
# [Omissis controlli fine ciclo e salto inattivi]
    move $ao, $to          # Passa l'indice al registro
                           # argomento
    jal get_student_gpa    # Ritorna Media in $fo e Stato
                           # in $v1
    beq $v1, $zero, next_threshold # Salta studenti senza
                           # esami
  
```

Dopo aver acquisito e memorizzato la soglia nel registro coprocessore `$f2`, il sistema itera sui record. Per ogni studente valido, invoca il *backend* per ricavare la media istantanea e ignora preventivamente i libretti vuoti per evitare errori di calcolo.

```

c.lt.s $fo, $f2          # (FPU) Compara: Media_Studente
                           # < Soglia?
    bc1t next_threshold  # Branch on Coprocessor 1
                           # True
  
```

Snodo logico della funzione. Delega il confronto al coprocessore matematico (`c.lt.s`): se la media corrente calcolata (`$f0`) è inferiore alla soglia richiesta (`$f2`), il

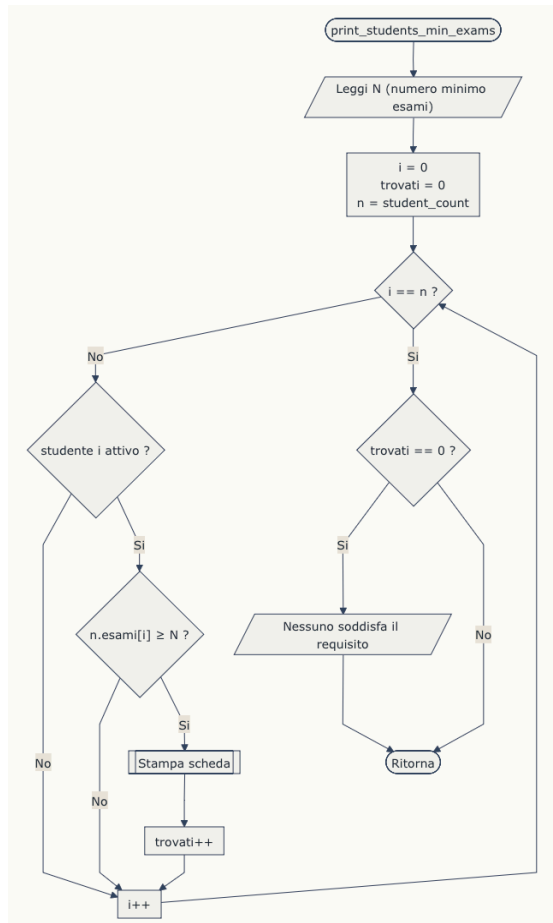
salto condizionato `bc1t` bypassa il blocco di stampa passando direttamente all'iterazione successiva.

```

move $ao, $to
print_student($ao)    # Stampa i dati dell'idoneo
addi $s4, $s4, 1      # Incrementa i match positivi
next_threshold:
addi $to, $to, 1      # i++
j threshold_loop
  
```

Se l'FPU non innesca il salto, lo studente soddisfa i criteri accademici. La macro di stampa viene invocata e un registro accumulatore (\$s4) tiene traccia del numero totale di occorrenze trovate per mostrare il riepilogo a fine ciclo.

3.3.12 Caso 12: Filtro per Numero Minimo di Esami (print_students_min_exams)



Esegue una ricerca quantitativa interrogando direttamente e linearmente la variabile di conteggio del libretto.

Analisi Strutturale del Codice Sorgente

```

chiedi_intero(min_exams_msg, $s3) # $s3 = target
minimo richiesto (N)
min_exams_loop:
# [Omissis controlli di fine loop e flag active]
sll $t1, $t0, 2      # Offset Word (i * 4)
la $t2, student_nexams
add $t2, $t2, $t1
lw $t3, 0($t2)      # $t3 = numero esami dello
studente corrente

```

Richiede il valore target N e lo salva in \$s3. Entrando nel loop di scansione, calcola l'offset a 32-bit e carica dal vettore `student_nexams` il numero di corsi effettivamente superati dallo studente in esame.

```

blt $t3, $s3, next_min_exam # Se n_esami < N, salta alla
cella successiva

```

```

move $a0, $t0
print_student($a0)      # Stampa lo studente idoneo
addi $s4, $s4, 1        # Contatore occorrenze++

```

```

next_min_exam:
addi $t0, $t0, 1      # i++
j min_exams_loop

```

Implementa il filtro con una singola istruzione hardware (**blt**): se il valore estratto dalla memoria è strettamente minore del target dell'utente, scarta il record. In caso contrario, lo studente viene visualizzato a schermo e conteggiato per il report finale.

3.3.13 Caso 13: Ordinamento per ID (sort_students_by_id)

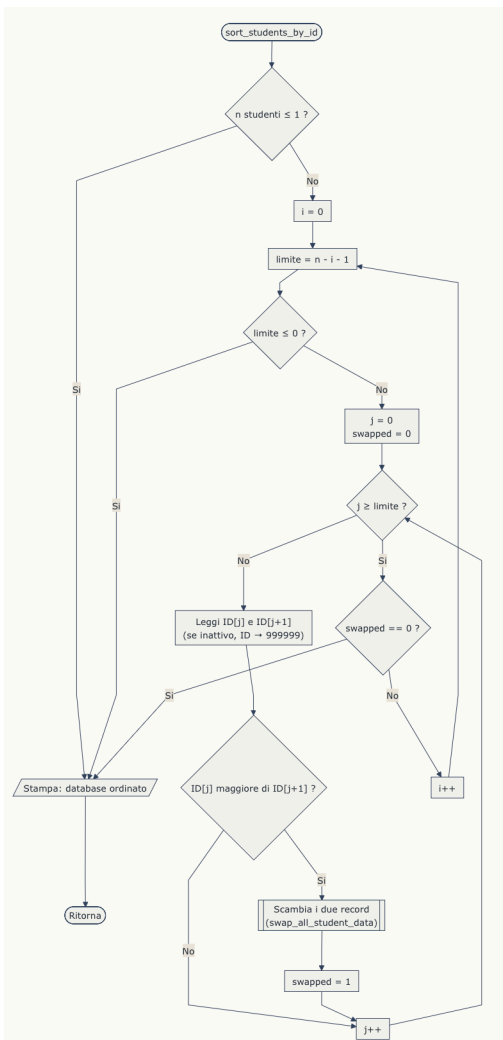
Implementa un algoritmo di **Bubble Sort ottimizzato** ($O(N^2)$) per riordinare l'intero database in senso crescente. Sfrutta routine di basso livello per scambiare fisicamente i blocchi di memoria.

Analisi Strutturale del Codice Sorgente

```

lw $t3, 0($t1)      # Estrae id[j]

```



bne \$t2, \$zero, active_j_ok # Se è attivo, salta l'assegnazione
li \$t3, 999999 # Se inattivo, assegna ID fittizio
altissimo
active_j_ok:

Descrizione: Estrazione e Validazione: Estrae l'identificativo e il flag di stato della cella corrente. Gestione Inattivi: Se rileva uno studente disattivato logicamente, il software assegna dinamicamente un ID virtuale altissimo (999999). Questo stratagemma costringe l'algoritmo di sorting a spingere il record verso il fondo della memoria.

ble \$t3, \$t6, no_swap # Se id[j] <= id[j+1], l'ordine è corretto
move \$a0, \$s1 # Passa indice j
move \$a1, \$t4 # Passa indice j+1
jal swap_all_student_data # Invoca lo scambiatore di array globale
li \$s4, 1 # Imposta flag swapped = 1 (Vero)

Descrizione: Confronto: Valuta due record adiacenti. Scambio Globale (Swap): Se il primo termine è maggiore del secondo, invoca la utility **swap_all_student_data** per invertire byte per byte tutti i vettori paralleli (anagrafica ed esami). Flag Sentinella: Il registro **\$s4** arresta il ciclo in anticipo se rileva una passata priva di scambi.

3.3.14 Caso 14: Ordinamento per Media (sort_students_by_gpa)

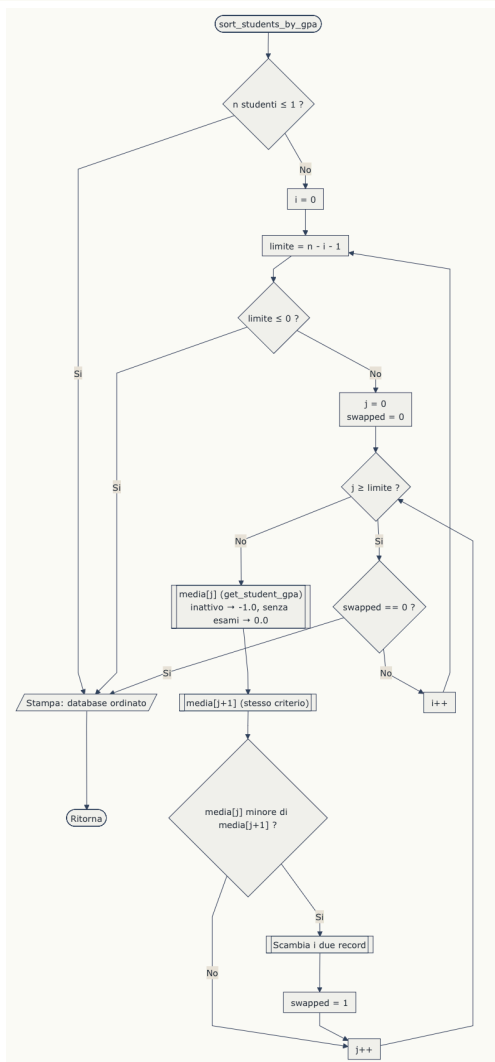
Riordina i record in senso decrescente valutando il rendimento accademico, demandando i confronti frazionari al coprocessore matematico (FPU).

Analisi Strutturale del Codice Sorgente

jal get_student_gpa # Ritorna media in \$fo
beq \$v1, \$zero, j_no_exams # Se attivo ma ha 0 esami, salta
mov.s \$f20, \$fo # Salva la vera media in \$f20
j calc_j_done
j_inactive:
li \$to, -1
mtc1 \$to, \$f20
cvt.s.w \$f20, \$f20 # Converti a float (-1.0)

Priorità di Calcolo: Invoca il calcolo del GPA e preserva in **\$f20** il valore esatto per i record attivi. Penalità Matematiche: Forza il valore a 0.0 per chi ha il libretto vuoto e applica un malus estremo di -1.0 (convertito in virgola mobile via **cvt.s.w**) per affondare in coda i record cancellati.

c.lt.s \$f20, \$f22 # Compara se Media(j) < Media(j+1)



```
bcif no_swap_gpa      # Se Falso, l'ordine è
corretto
```

```
# [Omissis invocazione swap_all_student_data]
```

Valutazione FPU: Delega il calcolo hardware all'unità in virgola mobile (c.lt.s). Trattandosi di ordine decrescente, la condizione di trigger si innesca solo se il primo termine è minore del secondo, richiamando la medesima procedura di Swap universale vista in precedenza.

3.3.15 Caso 15: Statistiche Globali (print_statistics)

Algoritmo ad alta efficienza che aggrega i dati computando medie multiple mediante una singola iterazione (complessità $O(N)$).

Analisi Strutturale del Codice Sorgente

```
# [Omissis caricamento età, esami e media negli
accumulatori $s3, $s4, $f20]
```

```
mtc1 $s2, $f1
```

```
cvt.s.w $f1, $f1      # $f1 = float(active_count)
```

```
# [Omissis stampa titoli]
```

```
mtc1 $s3, $fo
```

```
cvt.s.w $fo, $fo      # Converta sum_age in float
```

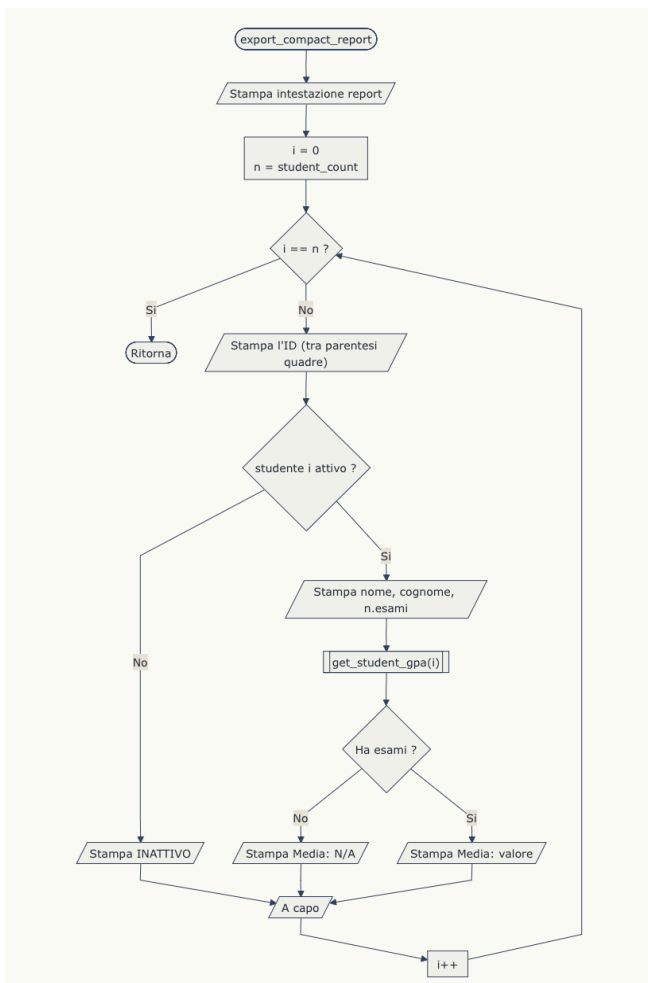
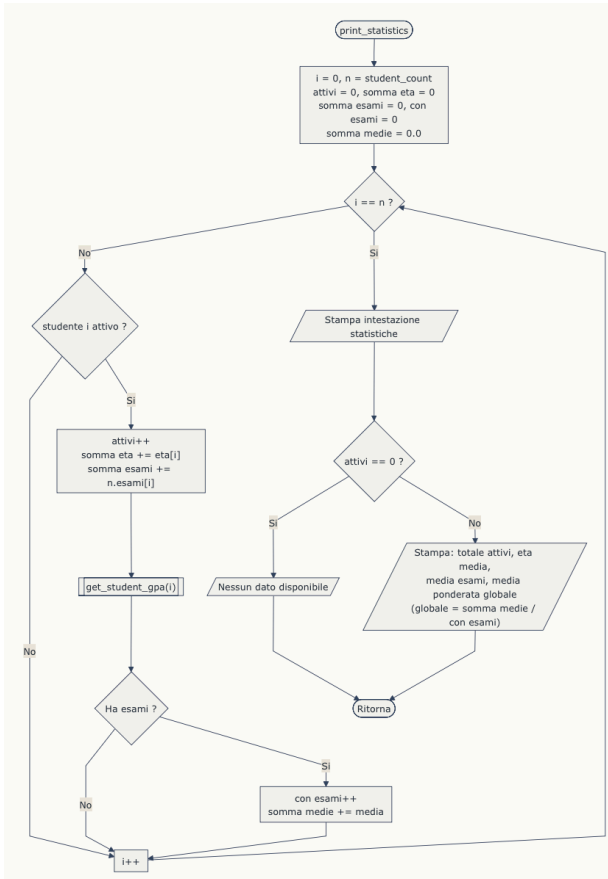
```
div.s $f12, $fo, $f1  # div_age = sum_age /
active_count
```

Accumulazione Dati: Scandisce il database sommando iterativamente i target nei registri (**\$s3** per età, **\$s4** per esami). Casting FPU: Converte i contatori interi in virgola mobile e utilizza l'istruzione **div.s** per eseguire la divisione esatta a precisione singola, formattando infine a video l'output statistico globale.

3.3.16 Caso 16: Report Compatto (export_compact_report)

Fornisce una sintesi tabulare estrapolando esclusivamente le chiavi primarie, bypassando i dettagli secondari non necessari.

Analisi Strutturale del Codice Sorgente




```

# [Omissis stampe ID]
lw $t2, 0($t1)      # Estrazione student_active
bne $t2, $zero, report_active
li $vo, 4
la $ao, rep_inactive # Stampa l'etichetta di stato "[INATTIVO]"
j report_next_line   # Salta i calcoli anagrafici e va a capo

```

Bypass Condizionato: Durante la scansione, interroga preventivamente il flag del record. Se identifica un utente sospeso, interrompe istantaneamente il recupero dei buffer alfabetici, stampa l'avviso "Inattivo" e forza l'avanzamento carrello (\n), garantendo velocità di esecuzione e simmetria visiva.

3.3.17 Caso 17: Stampa Esami Ricorsiva (recursive_print_exams)

Funzione di pregio ingegneristico che abbandona il classico ciclo iterativo per implementare un motore di estrazione basato sulla Ricorsione Pura.

Analisi Strutturale del Codice Sorgente

print_exams_recursive_helper:

```

beq $a1, $a2, recursive_base_case # CASO BASE:
Esami finiti

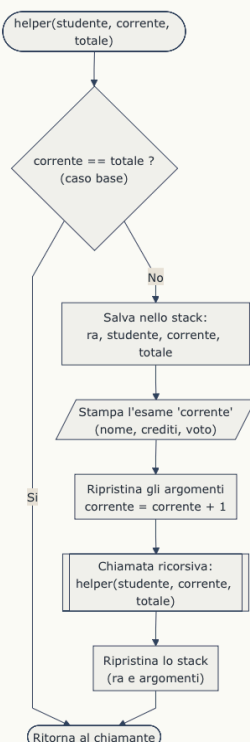
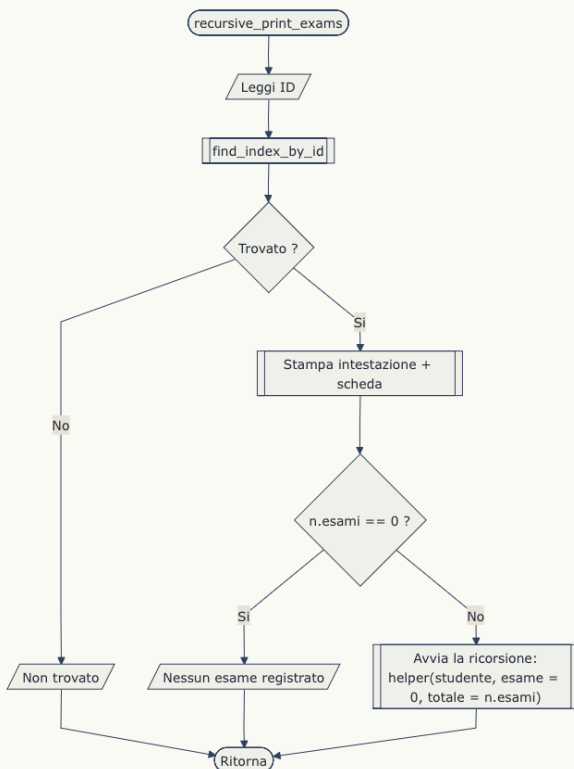
```

```

addi $sp, $sp, -16 # SALVATAGGIO NELLO
STACK
sw $ra, 0($sp)
sw $ao, 4($sp) # Indice studente
sw $a1, 8($sp) # Indice esame corrente
sw $a2, 12($sp) # Totale esami

```

Architettura del Nodo: Verifica istantaneamente il Caso Base (raggiungimento del totale esami). Push in Memoria: Esegue il salvataggio critico nello Stack, preservando l'indirizzo di ritorno **\$ra** e i registri argomento per evitare che le auto-invocazioni causino corruzioni sovrascrivendo i puntatori locali.



```

# [Omissis calcolo offset e stampe esame]
lw $ao, 4($sp) # Ripristino parametri per chiamata
lw $a2, 12($sp)
lw $a1, 8($sp)
addi $a1, $a1, 1 # Incremento indice esame
jal print_exams_recursive_helper # CHIAMATA RICORSIVA

```

Al termine dell'estrazione matriciale per l'esame corrente, ricarica i parametri originali dallo Stack, incrementa dinamicamente l'indice e innesca la ricorsione auto-chiamandosi tramite **jal**.

```

lw $ra, 0($sp) # RIPRISTINO E RISALITA (Pop)

```



```
# [Omissis Pop registri e disallocazione Stack]
recursive_base_case:
    jr $ra          # Torna alla chiamata precedente
```

Riavvolgimento (Pop): Quando il caso base arresta la discesa, la struttura inizia a riavvolgersi. Ogni frame esegue il Pop ripristinando progressivamente la RAM locale, risalendo le chiamate sino alla restituzione definitiva del controllo al Main.

3.4 funzioni utility

#	Funzione MIPS	Descrizione
1	<code>find_index_by_id(ID)</code>	Scorre linearmente l'array degli ID e restituisce l'indice assoluto del record, escludendo i nodi cancellati.
2	<code>get_student_gpa(indice)</code>	Computa la media pesata del libretto linearizzato eseguendo la divisione finale in virgola mobile nell'FPU.
3	<code>strip_newline</code>	Sanifica gli input testuali convertendo automaticamente il carattere di fine riga (<code>\n</code>) nel terminatore logico (<code>\0</code>) per garantire la validità delle stringhe.
4	<code>copy_string</code>	Trascrive byte per byte le stringhe in memoria bloccandosi in sicurezza al terminatore o al limite massimo strutturale di 19 caratteri per prevenire <i>overflow</i> .
5	<code>swap_bytes_memory</code>	Esegue lo scambio fisico a basso livello di blocchi contigui di byte tra due indirizzi RAM, ignorando il tipo di dato astratto sottostante.
6	<code>swap_all_student_data</code>	Trasla simultaneamente tutti i vettori paralleli di due record per preservare la coerenza architetturale durante le operazioni di ordinamento (es. <i>Bubble Sort</i>).

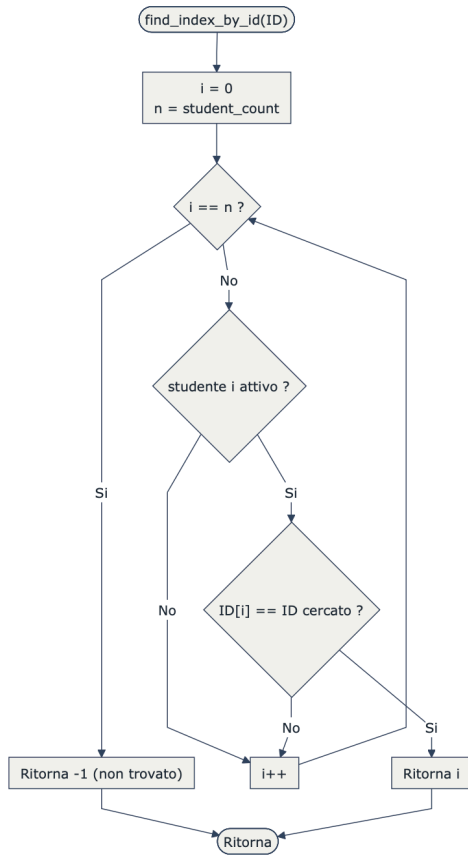
Al fine di garantire la massima modularità del software ed evitare la duplicazione del codice sorgente, le operazioni ricorrenti di basso livello e di calcolo sono state isolate in funzioni di *backend* dedicate. Per preservare la sintesi dell'elaborato e non estendere eccessivamente la trattazione, vengono analizzate nel dettaglio (mediante diagramma di flusso e frammenti di codice) esclusivamente le due routine principali del sistema, le quali governano l'accesso e la matematica dell'intero database. Le restanti utility minori vengono documentate sinteticamente in quanto basate su cicli standard di copia e manipolazione sequenziale della memoria.

3.4.1 Ricerca Indice per Identificativo (`find_index_by_id`)

Invocata prima di ogni operazione mirata su un singolo studente (modifica, calcolo media, stampa singola), questa routine astrae la logica di ricerca sequenziale dal main loop, restituendo l'indice fisico del record oppure un flag di errore. (usata nei casi 3, 5, 6, 7, 8, 10 e 17.)

Analisi Strutturale del Codice Sorgente

```
find_id_loop:
```



```

beq $t2, $t1, find_id_not_found # Se i == count, esce dal loop

```

--- CONTROLLO INTEGRITÀ ELIMINAZIONE LOGICA ---

```

# [Omissis calcolo offset in $t3]

```

```

lw $t5, o($t4) # Estrae student_active

```

```

beq $t5, $zero, find_id_next # Se active == 0, ignora e prosegui oltre

```

Scansione Lineare Filtrata: La funzione scorre in modo sequenziale il database. Prima ancora di confrontare le chiavi di ricerca, esegue un controllo critico sul flag **student_active**. Se rileva uno zero logico, l'istruzione beq bypassa i controlli successivi, impedendo all'algoritmo di generare "falsi positivi" su record che hanno subito un Soft-Delete.

Controllo ID Effettivo

```

lw $t5, o($t4) # Carica student_id[i]

```

```

beq $t5, $a0, find_id_success # Match trovato!

```

```

find_id_success:

```

```

# Salva l'indice i in $v0

```

```

move $v0, $t2

```

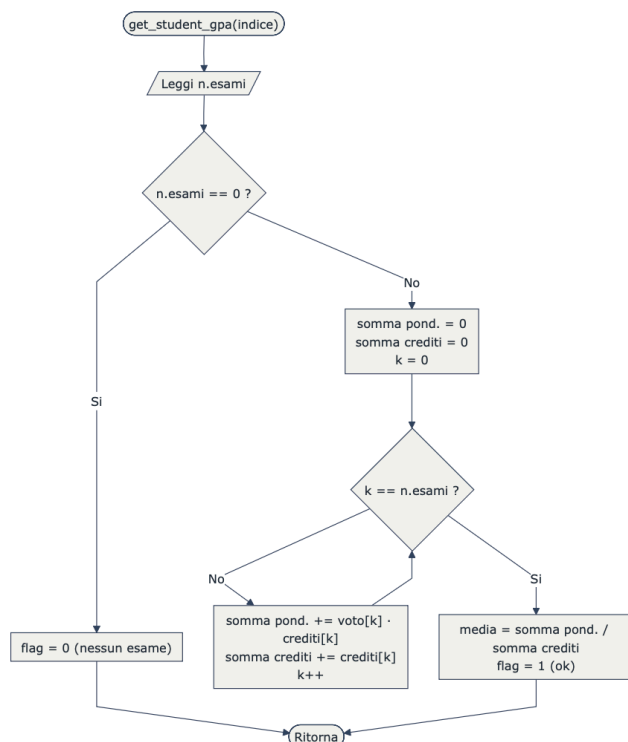
```

jr $ra

```

Confronto e Restituzione: Solo dopo aver confermato la validità del record, verifica se l'ID corrente combacia con quello richiesto dall'utente (\$a0). In caso di successo, instrada l'indice esatto nel registro di output \$v0 e riavvolge lo stack (jr \$ra). In caso di fallimento globale, la funzione termina impostando \$v0 a -1.

3.4.2 Calcolo Media Ponderata (get_student_gpa)



Costituisce il nucleo aritmetico avanzato dell'applicazione. Isola l'iterazione sulle matrici linearizzate degli esami ed eleva la precisione dei calcoli delegandoli al coprocessore matematico (FPU). (Usata dai casi 8, 9, 11, 14, 15 e 16.)

Analisi Strutturale del Codice Sorgente

```

get_gpa_loop:

```

```

    beq $t9, $s2, get_gpa_calc # Fine ciclo

```

```

    esami

```

```

    # [Omissis calcolo offset assoluto: ((i * 20) + k) * 4]

```

```

    lw $t3, o($t0) # $t3 = Crediti esame

```

```

    add $s3, $s3, $t3 # Accumulatore

```

```

    Somma Crediti

```

```

    # [Omissis estrazione voto in $t4]

```

mul \$t5, \$t3, \$t4	# Ponderazione (Voto * Crediti)
add \$s1, \$s1, \$t5	# Accumulatore Somma Ponderata

Accumulazione Scalare: Previene nativamente le eccezioni di divisione per zero bypassando il ciclo per gli studenti senza esami. Durante l'iterazione, calcola l'offset a 32-bit (srotolando la matrice fittizia 2D). Estrae voto e crediti, li moltiplica istantaneamente e popola due registri protetti (\$s1 per i punti totali, \$s3 per i crediti).

```
get_gpa_calc:
    mtc1 $s1, $fo
    cvt.s.w $fo, $fo          # Converta Somma Ponderata in float
    mtc1 $s3, $f1
    cvt.s.w $f1, $f1          # Converta Somma Crediti in float
    div.s $fo, $fo, $f1        # $fo = (Somma Ponderata) / (Somma Crediti)
    li $v1, 1                  # Imposta flag di successo
```

Casting e Divisione Hardware: Terminata l'accumulazione intera, il codice effettua il passaggio dati all'FPU (mtc1). Sfrutta l'istruzione cvt.s.w per convertire i totali nello standard a virgola mobile (IEEE 754) e innesca l'operatore hardware div.s per ottenere il quoziente esatto in \$f0. Contemporaneamente, setta un flag di validità in \$v1 per notificare al chiamante l'avvenuto calcolo.

3.5 funzioni macro (macros.asm)

Per alleggerire il codice sorgente nei vari casi d'uso e migliorare la leggibilità, le sequenze di istruzioni ricorrenti (come l'I/O con l'utente o il ricalcolo degli offset) sono state incapsulate in direttive .macro. Durante l'assemblaggio, MARS espande automaticamente queste chiamate sostituendole con il codice macchina effettivo.

#	Funzione MIPS	Descrizione
1	chiedi_intero(%msg_label, %reg_dest)	Accorpa le Syscall 4 e 5 per stampare a video un messaggio di prompt personalizzato e salvare l'input numerico dell'utente nel registro di destinazione.
2	chiedi_stringa(%msg_label, %indirizzo_dest, %lunghezza_max)	Unifica le Syscall 4 e 8 per richiedere un input testuale, salvandolo direttamente in RAM nel buffer specificato e rispettando il limite hardware di sicurezza.
3	cerca_studente_per_id(%id_reg, %not_found_label)	Incapsula l'invocazione alla funzione di backend find_index_by_id, deviando automaticamente l'esecuzione all'etichetta di errore fornita se il record non esiste (-1).
4	print_student(%indice)	Estrae e formatta a schermo l'intera anagrafica del record (ID, Nome, Cognome, Età, Anno), ricalcolando internamente a runtime tutti gli offset necessari.
5	stampa_linea_separatrice	Stampa una stringa decorativa preimpostata per migliorare la formattazione e la scansionabilità visiva dei report testuali a console.

3.6 Robustezza del Sistema e Prevenzione dei Fatal Crash

Oltre alle problematiche legate alla gestione del carattere di *newline*, la natura a basso livello dell'architettura MIPS ha richiesto l'implementazione sistematica di barriere di controllo software per impedire eccezioni hardware distruttive (crash dell'emulatore) o la corruzione invisibile dei dati in RAM.

1. **Prevenzione del Buffer Overflow (Sicurezza delle Stringhe):** L'acquisizione del Nome e Cognome dello studente tramite la macro `chiedi_stringa` impone un limite hardware invalicabile di 19 caratteri memorizzabili (passando il valore 20 nel registro `$a1` della Syscall 8). Questo impedisce la sovrascrittura accidentale delle Word contigue appartenenti agli array paralleli adiacenti (es. l'età dello studente o i flag di stato), preservando l'integrità strutturale della memoria.
2. **Protezione dalle Eccezioni di Divisione per Zero (FPU Trap):**
Nel motore di calcolo della media ponderata (`get_student_gpa`), prima di trasferire il controllo al Coprocessore Matematico (Coprocessor 1) per la divisione in virgola mobile, il codice esegue un controllo preliminare sul numero di esami sostenuti (`student_nexams`). Se il valore è pari a zero, il ciclo viene bypassato e viene restituito un flag logico di errore (`$v1 = 0`). Questo previene il sollevamento dell'eccezione hardware di *Division by Zero* nella Floating Point Unit, che causerebbe l'arresto immediato del programma.
3. **Validazione degli Ingressi Numerici (Anagrafica Coerente):**
Durante la creazione o modifica di un record, il software valida l'età dello studente rifiutando qualsiasi valore esterno al range logico [18, 100]. In caso di input non conforme, il sistema pulisce i registri temporanei e costringe l'utente a ripetere l'inserimento mediante un ciclo di controllo condizionale.

4. Conclusioni e Limiti Architettureali

La realizzazione dell'*University Student Information System* tramite l'emulatore MARS 4.5 ha dimostrato concretamente come sia possibile gestire un database completo pur operando a basso livello. Rispettando i rigidi vincoli strutturali imposti dal processore e dalla gestione fisica della CPU, il progetto implementa con successo funzionalità solitamente associate a linguaggi più avanzati, tra cui la gestione dinamica dei record (inserimento, lettura, modifica e cancellazione), l'ordinamento degli array e il calcolo delle medie in virgola mobile.

4.1 Bilancio delle Sfide Ingegneristiche Risolte

Le sfide più rilevanti affrontate e formalizzate nell'elaborato includono:

- **Disciplina dello Stack Pointer:** La cooperazione tra le macro riutilizzabili e le funzioni di backend ha richiesto una gestione rigorosa del frame di memoria tramite il registro `$sp`. La corretta conservazione dell'indirizzo di ritorno (`$ra`) e dei registri preservati (`$s0-$s7`) ha scongiurato il rischio di loop infiniti o corruzioni di contesto, raggiungendo il massimo livello di complessità nella gestione dei record degli esami tramite chiamate ricorsive self-invoking.
- **Ottimizzazione dei Cicli di CPU:** L'adozione della strategia di Soft-Delete (azzeramento del flag `student_active`), preferita allo shifting fisico dei blocchi di memoria dopo un'attenta analisi della traccia, ha permesso di mantenere un costo

computazionale pari a $O(1)$ per le operazioni di cancellazione, delegando il filtraggio dei record inattivi alle sole routine di scansione e reportistica.

4.2 Limiti Intrinseci e Sviluppi Futuri

Sebbene l'applicazione soddisfi pienamente i requisiti di modularità, efficienza e stabilità richiesti, essa risente dei limiti intrinseci dell'architettura hardware simulata:

- **Spreco Spaziale dovuto alla Allocazione Statica:** Non potendo fare affidamento su un gestore dinamico della memoria (come le funzioni `malloc/free` dei linguaggi di alto livello, la cui simulazione tramite la syscall `sbrk` avrebbe compromesso la stabilità strutturale del database parallelo), lo spazio in RAM è pre-allocato staticamente nella sezione `.data`. Questo comporta un consumo fisso di memoria anche in presenza di un database semivuoto.
- **Linearizzazione Rigida delle Matrici:** Lo "srotolamento" monodimensionale della matrice bidimensionale degli esami ha risolto il problema dell'assenza di strutture dati native in Assembly, ma costringe il processore a eseguire calcoli di offset moltiplicativi a runtime per ogni singolo accesso alle celle, gravando marginalmente sui cicli di clock.

In conclusione, l'adozione di una rigorosa architettura ad Array Paralleli, unita alla creazione di una libreria standardizzata di macro (`macros.asm`) e utility di servizio (`functions.asm`), ha permesso di simulare con successo le best-practice ingegneristiche dei linguaggi di alto livello, dimostrando che l'efficienza algoritmica prescinde dal grado di astrazione del linguaggio utilizzato.